

CSS Hero Section Layout & Full-Width Background

A step-by-step breakdown of full-bleed backgrounds and 1280px grid content alignment.

1. The 2-Layer Container Pattern

When building hero sections with full-screen animated backgrounds (like `ShapeGrid` or canvas shaders), you need two distinct layout layers:

```
+-----+
| .hero-section (Outer Layer: 100% Width, Full-Screen Background) |
| |
| | +-----+
| | | .hero-content (Inner Layer: max-width: 1280px, Centered) |
| | | |
| | | | a.niche
| | | | PRODUCT DESIGNER
| | | | /anish/
| | | +-----+
| | +-----+
| +-----+
```

- **Outer Section** (`.hero-section`): Spans `width: 100%; max-width: 100vw;` so the background grid fills the entire screen from edge to edge.
- **Inner Content Box** (`.hero-content`): Constrained to `max-width: 1280px; margin: 0 auto;` to align text perfectly with the site grid.

2. Flexbox Alignment: `flex-start` vs `left`

In CSS Flexbox specifications:

- `align-items: left;` is **invalid syntax** in CSS flexbox.
- `align-items: flex-start;` is the standard rule to align stacked flex elements to the left margin.

3. CSS Implementation

```
/* 1. Outer Section: Full Viewport Width for Background Effect */
.hero-section {
  position: relative;
  width: 100%;
  max-width: 100vw;
  display: flex;
  flex-direction: column;
  align-items: center;
  overflow: hidden;
}
```

```
/* 2. Inner Box: 1280px Centered Container with Left Text */
.hero-content {
  position: relative;
  z-index: 1;
  display: flex;
  flex-direction: column;
  align-items: flex-start; /* Aligns flex children to left */
  text-align: left;
  max-width: 1280px;      /* Standard site grid width */
  width: 100%;
  margin: 0 auto;         /* Horizontally centers 1280px container */
  padding: 0 1.5rem;      /* Side gutters for smaller screens */
  box-sizing: border-box;
}
```